Day 115 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 17, Day 4 reference

Watch: StatQuest: K-nearest neighbors, Clearly Explained | The Curse of Dimensionality (Week 03-01) (Zubair Khalid)

`day115_k_nearest_neighbors.py`, segment by segment, in the order `main()` uses things. kNN and PCA are new; Day 112's scaling and splitting are imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: k = neighbours · d = dimensions · D = the table of distances · U, Σ, V = the parts of the singular value decomposition

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day112_support_vector_machines as d112
standardise, stratified_split, d99 = d112.standardise, d112.stratified_split, d112.d99
KS = [1, 3, 5, 7, 9, 15, 25, 51]
DIMS = [1, 2, 5, 10, 20, 50, 100, 200, 500, 1000]
NOISE = [0, 16, 64, 256, 1024]
COMPS = [2, 5, 10, 15, 20, 30, 40, 64]
SPLITS = 5
```

- `standardise, stratified_split, d99 = d112.standardise, ...` — Day 112's helpers, and Day 99's bootstrap through it.
- `DIMS = [1, 2, 5, ..., 1000]` — roughly evenly spaced on a log scale.

Segment 1 — kNN and PCA

knn_predict: distances, the k nearest, a vote

MATH

$D = |x|^2 + |z|^2 - 2x \cdot z$ *for every pair at once*

CODE

```
def sqdist(A, B):
    return (A ** 2).sum(1)[:, None] + (B ** 2).sum(1)[None, :] - 2 * A @ B.T
def knn_predict(Xtr, ytr, X, k, chunk=500):
    n_classes = int(ytr.max()) + 1; out = []
    for s in range(0, len(X), chunk):
        D = sqdist(X[s:s + chunk], Xtr)
        idx = np.argpartition(D, k - 1, axis=1)[:, :k] if k < len(ytr) else
            np.tile(np.arange(len(ytr)), (len(D), 1))
        votes = np.zeros((len(D), n_classes))
        for j in range(k):
            votes[np.arange(len(D)), ytr[idx[:, j]]] += 1
        d = np.take_along_axis(D, idx, 1)
        tie = (votes == votes.max(1, keepdims=True)).sum(1) > 1
        for r in np.where(tie)[0]:
            best = np.where(votes[r] == votes[r].max())[0]
            order = idx[r][np.argsort(d[r])]
            out_r = next(ytr[i] for i in order if ytr[i] in best); votes[r, out_r] += 0.5
        out.append(np.argmax(votes, 1))
    return np.concatenate(out)
```

- `np.argpartition(D, k - 1, axis=1)[:, :k]` — the k smallest per row, without sorting the whole row: faster than argsort.
- `votes[np.arange(len(D)), ytr[idx[:, j]]] += 1` — fancy indexing with two arrays: one vote per row in one step.
- `np.take_along_axis(D, idx, 1)` — the distances at those k positions, row by row.
- `next(ytr[i] for i in order if ytr[i] in best)` — the first item of a generator: the class of the nearest neighbour among the tied classes.

pca_fit: the singular value decomposition

MATH

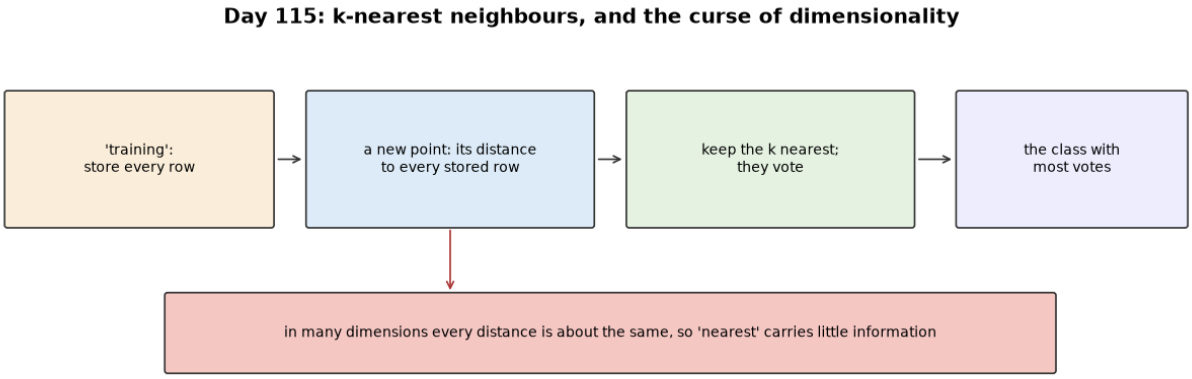
$X - \mu = U \Sigma V^T$; *variance share of direction* $i = \sigma_i^2 / \Sigma \sigma^2$

CODE

```
def pca_fit(X, k):
    mu = X.mean(0); _, s, Vt = np.linalg.svd(X - mu, full_matrices=False)
    return mu, Vt[:k], (s ** 2) / (s ** 2).sum()
```

- `np.linalg.svd(X - mu, full_matrices=False)` — the economy decomposition: only as many directions as needed.
- `Vt[:k]` — the first k rows: the directions of largest variance.
- `(X - mu) @ V.T` — (used by the demos) each row's coordinates along those directions.

SYSTEM DIAGRAM



kNN, and where it breaks.

Segment 2 — The experiments

Concentration, inside demo_concentration

CODE

```
rng = np.random.RandomState(d); P = rng.uniform(size=(1000, d)); Q = rng.uniform(size=(100, d))
D = np.sqrt(np.maximum(sqdist(Q, P), 0)); lo, hi, mu = D.min(1), D.max(1), D.mean(1)
rows.append((d, np.median((hi - lo) / lo), np.median(lo / mu)))
```

- `np.sqrt(np.maximum(sqdist(Q, P), 0))` — true distances; maximum guards against rounding below zero.
- `D.min(1), D.max(1), D.mean(1)` — per query: nearest, farthest and average.
- `np.median(...)` — a typical query, not swayed by extremes.

Noise dimensions, inside demo_noise

CODE

```
Z = np.hstack([X, np.random.RandomState(s).uniform(0, 16, (len(y), k))]); tr, te =
    stratified_split(y, 0.3, s)
acc.append(np.mean(knn_predict(Z[tr], y[tr], Z[te], 5) == y[te]))
D = np.sqrt(np.maximum(sqdist(Z[te][:100], Z[tr]), 0)); ratio.append(np.median(D.min(1) /
    D.mean(1)))
```

- `np.hstack([X, noise])` — the noise columns are appended to the 64 pixels.
- `Z[te][:100]` — the first 100 test images, enough to measure the ratio.

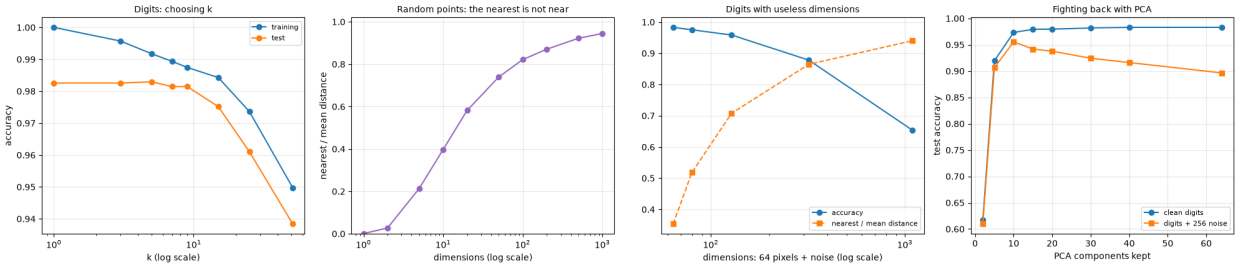
Timing, inside demo_pca

CODE

```
for n in (250, 500, 1000, 1257):
    sub = tr[:n]; t0 = time.perf_counter()
    for _ in range(5):
        knn_predict(X[sub], y[sub], X[te], 5)
    print(f"  {n:>5} training images: predicting {len(te)} test images takes {(time.perf_counter()
        - t0) / 5 * 1000:.1f} ms (mean of 5, within this run)")
```

- `sub = tr[:n]` — the first n training rows.
- `time.perf_counter()` — a precise wall clock; the mean of 5 runs is reported.

REAL GRAPH



Real graph from the script: k, concentration, noise dimensions and PCA.

Quick Syntax Glossary — new or reinforced constructs

- `np.argpartition(a, k - 1, axis=1)` — the k smallest per row, unordered.
- `np.take_along_axis(a, idx, 1)` — values at given positions, row by row.
- `np.linalg.svd(A, full_matrices=False)` — the singular value decomposition.
- `next(generator)` — the first item a generator produces.
- `np.hstack(list)` — join arrays side by side.
- `np.tile(a, (r, 1))` — repeat a row r times.

End of Day 115 syntax reference. For the experiments and what they showed, see the Day 115 complete guide.